

Minimum funkcji jednej zmiennej

fminbnd

Składnia

x = **fminbnd**(fun,x1,x2)

x = **fminbnd**(fun,x1,x2,options)

[**x**,**fval**] = **fminbnd**(...)

[**x**,**fval**,**exitflag**] = **fminbnd**(...)

[**x**,**fval**,**exitflag**,**output**] = **fminbnd**(...)

Wartości wskaźnika „flaga” w zależności od przyczyny zatrzymania algorytmu

Wartości wskaźnika „flaga”	Przyczyna zatrzymania algorytmu polecenia „fminbnd”
1	Funkcja zbieżna do rozwiązania x
0	Osiągnięto założoną maksymalną liczbę iteracji lub obliczeń wartości funkcji celu
-1	Algorytm zatrzymany przez funkcję zewnętrzną „output” lub „plot”
-2	Granice są niespójne, co oznacza $x1 > x2$

Przykład.

Funkcja $f(x) = (x - 2)^2 - \sin(x) + 1$

Narysować wykres tej funkcji w przedziale $[0, 3\pi]$.

Znaleźć minimum tej funkcji w przedziale $[0, 3\pi]$.

Podać wartość funkcji w tym punkcie x (minimum).

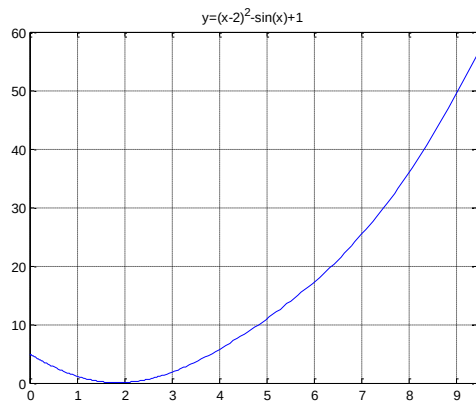
Wyświetlić informacje o poszczególnych iteracjach

Tworzymy plik: z3.m

%Funkcja z3

```
function y=z3(x)  
y=(x-2).^2-sin(x)+1;  
end
```

```
>> fplot(@z3,[0,3*pi])  
>> grid on  
>> title('y=(x-2)^2-sin(x)+1')
```



```
>> [x,fmin,flaga]=fminbnd('z3',0,3*pi)
```

```
x =  
    1.8582
```

```
fmin =  
    0.0611
```

```
flaga =  
    1
```

Możemy dołączyć do obliczeń informacje o kolejnych iteracjach, podając parametry **optimset**: Display + iter – wyświetla informacje o kolejnych iteracjach (ilość obliczeń wartości funkcji; bieżąca wartość poszukiwanego minimum; wartość funkcji w tym punkcie - $f(x)$, stosowana metoda):

Główne parametry funkcji **optimset**

Parametr	Wartość	Opis
'Display'	'off'	Bez wyświetlania wyników
	'iter'	Wyświetlanie wyników po każdej iteracji
	'notify'	Wyświetlanie wyników w razie konieczności
	'final'	Wyświetlanie ostatecznego rozwiązania wraz z informacją o warunkach jego wyznaczenia
'MaxFunEvals'	Dodatnia liczba całkowita	Maksymalna, dozwolona liczba obliczeń wartości funkcji
'MaxIter'	Dodatnia liczba całkowita	Maksymalna, dozwolona liczba iteracji
'TolFun'	Liczba	Dokładność wykonywania obliczeń funkcji
'TolX'	Liczba	Dokładność wykonywania obliczeń argumentu X lub kryterium zatrzymania algorytmu
'PlotFcns'	@optimplotx	Podaje na wykresie bieżącą wartość obliczonego punktu optymalnego
	@optimplotfval	Podaje na wykresie wartości optymalizowanej funkcji w poszczególnych iteracjach

	@optimplotfunccount	Podaje na wykresie liczbę obliczeń wartości funkcji w poszczególnych iteracjach (nie jest dostępna dla polecenia <i>fzero</i>)
--	---------------------	---

Zmiana sposobu wyświetlania informacji oraz dokładności obliczeń argumentu **x**

```
>>[x,fun,flaga]=fminbnd('z3',0,3*pi,optimset('Display','iter',...
'TolX',1e-15))
```

Func-count	x	f(x)	Procedure
1	3.59994	4.00229	initial
2	5.82483	16.0718	golden
3	2.22489	0.256974	golden
4	1.09729	0.924905	parabolic
5	1.88461	0.0621518	parabolic
6	1.86127	0.0611377	parabolic
7	1.85801	0.0611242	parabolic
8	1.85824	0.0611242	parabolic
9	1.85825	0.0611242	parabolic
10	1.85825	0.0611242	parabolic
11	1.85825	0.0611242	parabolic

Optimization terminated:
the current x satisfies the termination criteria using
OPTIONS.TolX of 1.000000e-015

x =
1.8582

fun =
0.0611

flaga =
1

Poszukiwanie minimum globalnego

Funkcja:

$$f(x) = e^{-0,1x} \cdot \cos x$$

$$x \in \langle 0, 20 \rangle$$

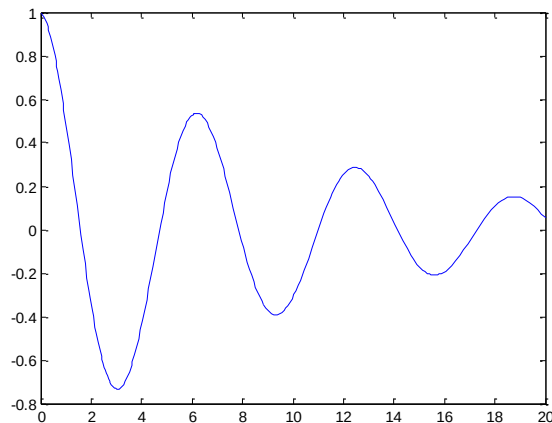
```
%funkcja fk1
```

```
%
```

```
function y=fk1(x)
```

```
y=exp(-.1*x) .* cos(x) ;
```

```
end
```



Tworzymy skrypt np. pod nazwą **fk1a.m**

Dodatkowo:

- zwiększamy dokładność obliczeń argumentu funkcji do 10^{-15} .
- ustawiamy maksymalną liczbę iteracji na 300 oraz maksymalną liczbę obliczeń wartości funkcji celu na 500.

```
%Poszukiwanie minimum globalnego w przedziale <0,20>
```

```
%zbiór fk1a
```

```

%przedziały z wykresu
a0=0;
a1=6;
a2=12;
a3=18;
a4=20;
%rozwiązanie
%opcje optymalizacji
options=optimset('MaxIter',300,'MaxFunEvals',500,'TolX',1e-15);
[x1,f1,flaga1] = fminbnd('fk1',a0,a1,options);
[x2,f2,flaga2] = fminbnd('fk1',a1,a2,options);
[x3,f3,flaga3] = fminbnd('fk1',a2,a3,options);
[x4,f4,flaga4] = fminbnd('fk1',a3,a4,options);

%początek przedziału
x0=a0;
f0=fk1(a0);
%koniec przedziału
x5=a4;
f5=fk1(a4);

%zestawienie wyników
x=[x0;x1;x2;x3;x4;x5];
f=[f0;f1;f2;f3;f4;f5];
a=[a0;a0;a1;a2;a3;a4];
b=[a0;a1;a2;a3;a4;a4];
flaga=[1;flaga1;flaga2;flaga3;flaga4;1];
clc
%prezentacja wyników
k=[x,f,a,b,flaga];
zestawienie_obliczen=table(x,f,a,b,flaga)
kk=sortrows(k,2); %sortowanie macierzy według kolumny nr 2 -"f(x)"
%kk=sortrows(k,2,'ascend');
x=kk(:,1);
f=kk(:,2);
a=kk(:,3);
b=kk(:,4);
flaga=kk(:,5);
zestawienie_uporzadkowane=table(x,f,a,b,flaga)
xmin=kk(1,1) %punkt minimum
fmin=kk(1,2)
%fmin=min(f) %minimalna wartość wektora f (wartości funkcji)

```

```

flaga=kk(1,5)
dokladnosc=options.TolX
maksymalna_liczba_iteracji=options.MaxIter
maksymalna_liczba_obliczen_wartosci_funkcji=options.MaxFunEvals

```

Zapisujemy skrypt pod nazwą np. fk1a.m i uruchamiamy go:

```
>>fk1a
```

```
zestawienie_obliczen =
```

6×5 table

x	f	a	b	flaga
0	1	0	0	1
3.0419	-0.73406	0	6	1
9.3251	-0.39161	6	12	1
15.608	-0.20892	12	18	1
20	0.055234	18	20	1
20	0.055228	20	20	1

```
zestawienie_uprzadkowane =
```

6×5 table

x	f	a	b	flaga
3.0419	-0.73406	0	6	1
9.3251	-0.39161	6	12	1
15.608	-0.20892	12	18	1
20	0.055228	20	20	1
20	0.055234	18	20	1
0	1	0	0	1

```

xmin =
3.0419

```

```
fmin =  
    -0.7341  
  
flaga =  
    1  
  
dokładność =  
    1.0000e-15  
  
maksymalna_liczba_iteracji =  
    300  
  
maksymalna_liczba_obliczen_wartosci_funkcji =  
    500  
  
>>
```

Poszukiwanie maksimum globalnego

Funkcja:

$$f(x) = e^{-0,1x} \cdot \cos x$$

$$x \in \langle 0, 20 \rangle$$

Modyfikujemy funkcję celu, mnożąc ją przez -1, i zapisujemy np. pod nazwą fk2.m

```
%funkcja fk2  
%  
function y=fk2(x)  
y=exp(-.1*x) .* cos(x) ;  
y=-y;  
end
```


Tworzymy skrypt np. pod nazwą `fk2a.m`, wykorzystując funkcję celu pomnożoną przez -1:

`%Poszukiwanie maksimum globalnego w przedziale <0,20>`
`%zbiór fk2a`

`%przedziały z wykresu`

`a0=0;`

`a1=6;`

`a2=12;`

`a3=18;`

`a4=20;`

`%rozwiązanie`

`options=optimset('MaxIter',300,'MaxFunEvals',500,'TolX',1e-15);`

`[x1,f1,flaga1] = fminbnd('fk2',a0,a1,options);`

`[x2,f2,flaga2] = fminbnd('fk2',a1,a2,options);`

`[x3,f3,flaga3] = fminbnd('fk2',a2,a3,options);`

`[x4,f4,flaga4] = fminbnd('fk2',a3,a4,options);`

`%początek przedziału`

`x0=a0;`

`f0=fk2(a0);`

`%koniec przedziału`

`x5=a4;`

`f5=fk2(a4);`

`%zestawienie wyników`

`x=[x0;x1;x2;x3;x4;x5];`

`f=[f0;f1;f2;f3;f4;f5];`

`a=[a0;a0;a1;a2;a3;a4];`

`b=[a0;a1;a2;a3;a4;a4];`

`flaga=[1;flaga1;flaga2;flaga3;flaga4;1];`

`clc`

`%Powrót do rzeczywistych wartości funkcji celu`

`f=-f;`

`clc`

`%prezentacja wyników`

`k=[x,f,a,b,flaga];`

`zestawienie_obliczen=table(x,f,a,b,flaga)`

`kk=sortrows(k,-2); %sortowanie macierzy według kolumny nr 2 - malejące wartości "f(x)"`

`%kk=sortrows(k,2,'descend');`

`x=kk(:,1);`

`f=kk(:,2);`

```

a=kk(:,3);
b=kk(:,4);
flaga=kk(:,5);
zestawienie_uporzadkowane=table(x,f,a,b,flaga)
xmax=kk(1,1) %punkt minimum
fmax=kk(1,2)
%fmax=max(f) %maksymalna wartość wektora f (wartości funkcji)
flaga=kk(1,5)
dokladnosc=options.TolX
maksymalna_liczba_iteracji=options.MaxIter
maksymalna_liczba_obliczen_wartosci_funkcji=options.MaxFunEvals

```

Zapisujemy skrypt pod nazwą np. fk2a.m i uruchamiamy go:

```
>>fk2a
```

zestawienie_obliczen =

6×5 table

x	f	a	b	flaga
0	1	0	0	1
5.7869e-05	0.99999	0	6	1
6.1835	0.53616	6	12	1
12.467	0.28603	12	18	1
18.75	0.1526	18	20	1
20	0.055228	20	20	1

zestawienie_uprzadkowane =

6×5 table

x	f	a	b	flaga

0	1	0	0	1
5.7869e-05	0.99999	0	6	1
6.1835	0.53616	6	12	1
12.467	0.28603	12	18	1
18.75	0.1526	18	20	1
20	0.055228	20	20	1

xmax =
0

fmax =
1

flaga =
1
dokładność =
1.0000e-15

maksymalna_liczba_iteracji =
300

maksymalna_liczba_obliczen_wartosci_funkcji =
500

>>